

Calling vetted calculation modules from a job file

A simply supported roof beam, analysed over the ultimate load combinations of AS/NZS 1170.0 and designed for flexure and shear to AS 3600:2018 — written the way you would run a real job.

WRITTEN FOR	An engineer who reads design codes fluently and Python slowly. No prior Python project experience assumed.
LIBRARY	<code>astruct</code> 0.1.0 — the toolkit in this repository
WORKED EXAMPLE	Job 25-0142 Northbank Depot, roof beam B1
EVERY PANEL	Real output. Terminal panels are captured stdout; code panels are read out of the committed files at build time.
STATUS	Not verified for issue. This teaches the workflow. It makes no claim that any number the library produces is correct — that validation is yours to do.

1 What you are going to build

A folder for one job, holding two short files you wrote, that together produce a signed calculation report for a beam. Neither file contains a clause of AS 3600. Every code equation comes from a library that lives elsewhere, is version controlled, and is tested.

That separation is the whole idea. A spreadsheet tangles three things with completely different lifetimes:

What it is	Changes	Should live
Code equations — ϕM_{uo} , k_v , θ_v	When the Standard is amended	In one vetted, tested library
The job — spans, loads, grades, the client	Every job	In a job folder
The decision — 350×650 , 3-N24	Every iteration	In one line you can read aloud

Re-using spreadsheet equations means copying the job with them — and that copy is now a second, unchecked version of AS 3600. Putting the equations in an importable package and the job in a script that imports it is what stops the copying. That is what `import` is really for.

Where this sits in the ASET framework

Ferster's six components are also the order a real calculation goes in. When you are lost in the code, "which of the six am I in?" usually unsticks it.

#	ASET component	In this job	Where the code lives
1	Reference data	C32 concrete, N24 bar areas	<code>austruct.materials</code>
2	Project data	Job record → ψ factors → combinations; wind	<code>austruct.project</code> , <code>austruct.loads</code> , <i>your job_data.py</i>
3	Fast demand calculation	7 combinations, analysed and enveloped	<code>austruct.analysis</code>
4	Design documentation	350×650 C32 B0T 3-N24 ...	<code>austruct.design_documentation</code>
5	Design verification	Flexure and shear to AS 3600:2018	<code>austruct.design.as3600</code>
6	Reporting	The signed audit document	<code>austruct.report</code>

ONE CAVEAT BEFORE YOU START

Every module in this library reports its status as `UNVERIFIED`, and every report it produces is stamped **NOT VERIFIED FOR ISSUE**. The constants are written but not independently checked against the printed Standards. This tutorial teaches the workflow; validating the numbers is the engineer's job, and part 4 shows you where to look to do it.

2 How Python finds other people's code

A **module** is a file. A **package** is a folder of them. The dots in an import are folder separators.

So `from austruct.design import as3600` reads as: go into `austruct/design/` and give me `as3600`. After that, `as3600.check_flexure(...)` calls a function in a file you did not write, did not copy, and cannot edit by accident while typing a span.

You write	You then use	Use it when
<code>import austruct</code>	<code>austruct.design.as3600.check_flexure(...)</code>	Rarely too long to read
<code>from austruct.design import as3600</code>	<code>as3600.check_flexure(...)</code>	Most the time The code still name the standard
<code>from austruct.design.as3600 import check_flexure</code>	<code>check_flexure(...)</code>	Sparingly — also it does not say AS 3600 rather than 5100

NEVER USE IMPORT *

`from austruct.design.as3600 import *` pulls every name into your file at once. It is the equivalent of pasting someone's whole spreadsheet into yours: you can no longer tell which numbers are yours, and a rename upstream silently changes your results. In a calculation file that is a traceability defect, not a style preference.

Where Python actually looks

- 1 **The folder of the script you ran** — not the folder you are standing in. This is why `beam_B1.py` can say `import job_data` with no setup: they are siblings.
- 2 **The site-packages of the environment you are in** — where `pip install` puts things, and how `austruct` is found from anywhere.
- 3 The standard library, and places you will not need to think about.

Both errors below are that list failing. First, the library is not in this environment because the environment was never activated:

```
bash

$ python beam_B1.py          # note: no (.venv) in the prompt
Traceback (most recent call last):
  File "/home/user/EngineeringEngine/tutorials/01_simply_supported_beam/job/beam_B1.py", line 15, in
<module>
    from austruct.analysis import UDL, PointLoad, SelfWeight, analyse_combinations, simply_supported
ModuleNotFoundError: No module named 'austruct'
```

Screenshot 2.1 — no `(.venv)` in the prompt. The fix is `source .venv/bin/activate`, not editing the file.

Second, your own module is not found because you are one directory too high — then the same import working one folder down:

```
bash

(.venv) $ cd ~/jobs/25-0142-northbank          # one level above the script
(.venv) $ python -c "import job_data"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'job_data'

(.venv) $ cd job
(.venv) $ python -c "import job_data; print(job_data.SPAN)"
7200.0
```

Screenshot 2.2 — `import job_data` is not magic. It works because `job_data.py` sits beside the file being run.

THE RULE TO MEMORISE

Library code gets installed. Job code sits next to the script. If you find yourself writing `sys.path.append(".././my_calcs")` to reach your own calculations, that is the signal those calculations have grown up and want to be an installed package too.

3 Setting up, from nothing

Two folders, and they must be different folders. One holds the library. One holds the job.

A *virtual environment* is a private copy of Python for one project, so that upgrading a package for this job cannot silently change last year's answers. Create it once in the library folder and activate it every time you open a terminal; the `(.venv)` in the prompt is the confirmation.

```
bash — in the library folder

$ python3 -m venv .venv
$ source .venv/bin/activate
(.venv) $ pip install -e ".[dev,plots,report,tools]"
Attempting uninstall: austruct
Found existing installation: austruct 0.1.0
Uninstalling austruct-0.1.0:
  Successfully uninstalled austruct-0.1.0
Successfully installed austruct-0.1.0
```

Screenshot 3.1 — `pip install -e` installs it *editable*: site-packages gets a pointer to the source folder rather than a copy, so a library fix is live immediately. The bracketed names are optional extras.

Before writing anything, prove it worked. The first command shows *which* copy you are about to use; the second runs the library's own tests — the evidence that the thing you are about to trust still behaves as its author intended.

```
bash

(.venv) $ python -c "import austruct; print(austruct.__version__, austruct.__file__)"
0.1.0 /home/user/EngineeringEngine/src/austruct/__init__.py

(.venv) $ pytest -q
799 passed, 3 skipped in 173.67s (0:02:53)
```

Screenshot 3.2 — the path is the source tree, not a copy: that is what *editable* means. 799 passing tests is not proof the clauses are right, but it is proof nothing has broken since they were written.

Then the job folder. One folder per *job*, not per member: B1, B2 and the transfer slab share a project record and a load schedule, and duplicating those is how they drift apart.

```

(.venv) $ tree ~/jobs/25-0142-northbank

25-0142-northbank/
|-- job/
|   |-- job_data.py
|   |-- beam_B1.py
|-- outputs/
|   |-- B1_diagrams.png
|   |-- B1_envelope.png
|   |-- B1_section.png
|   |-- B1_report.html
|   |-- B1_report.md

```

<- ONE folder per job. Not one per beam.
 <- the job: geometry, loads, wind, the record
 <- the member: analysis + design + report
 <- everything the run produces, nothing you type

```

(.venv) $ tree ~/code/engineeringengine -L 2 # the LIBRARY -- a different folder

engineeringengine/
|-- pyproject.toml
|-- src/austruct/
|   |-- materials/
|   |-- project/ loads/
|   |-- analysis/ structures/
|   |-- design_documentation/
|   |-- design/ sections/
|   |-- report/
|-- tests/

```

[1] reference data
 [2] project data
 [3] fast demand calculation
 [4] design documentation
 [5] design verification
 [6] reporting
 799 passing

Screenshot 3.3 — job folder above, library folder below; only one of them is installed. `outputs/` holds only generated files, so you can delete it and re-run to get it back. If you cannot, something in there was hand-edited.

4 From a bare equation to a vetted module

This part explains why the repository is shaped the way it is. You are not going to build these modules — they exist. But you should recognise the shape, because it is what makes them safe to call, and it is what you will validate.

The equation is the small part

The concrete compressive resultant of AS 3600 Cl 8.1.3 is three lines of arithmetic. In a script you would write it once and move on. In the library it is `rc_common/stress_block.py`, and the arithmetic is still three lines:

```
src/austruct/design/rc_common/stress_block.py
```

```
80     block_depth = min(concrete.gamma * dn, geometry.D)
81     area = geometry.area_above(block_depth)
82     force = concrete.alpha2 * concrete.fc * area
83     centroid = geometry.centroid_above(block_depth)
84
```

Screenshot 4.1 — $\alpha_2 f'_c$ over γd_n , exactly as printed in the Standard. Nothing clever happens here, and nothing should.

That arithmetic lives in `rc_common/`, shared by every standard. One layer up, `as3600/flexure.py` wraps it in the part that earns its keep: a docstring naming the clauses it implements and the range it is valid over, so a reviewer can check both without reading the code.

```
src/austruct/design/as3600/flexure.py
```

```
144     """Ultimate flexural capacity of a reinforced concrete section.
145
146     Basis
147     ----
148     AS 3600:2018:
149
150     - Cl 8.1.2 -- ultimate strength in bending, plane sections
151     - Cl 8.1.3 -- rectangular stress block, ``alpha_2`` and ``gamma``
152     - Cl 8.1.5 -- ductility limit ``k_uo <= 0.36``
153     - Cl 8.1.6.1 -- minimum strength, ``M_uo >= 1.2 M_cr``
154     - Table 2.2.2 -- capacity reduction factor as a function of ``k_uo``
155
156     Envelope
157     -
158     20 MPa <= f'c <= 100 MPa. Non-prestressed. Pure bending, no axial force.
159     Compression at the top of the section as supplied.
160
161     [UNITS] Returns moments in N.mm; the report displays them in kN.m.
162
```

Screenshot 4.2 — the AS 3600 wrapper around that mechanics, one layer up. `Basis` says which clauses. `Envelope` says the range it is valid over. Those two sections are the difference between an equation and a module you can hand to a checker.

Three stages, and why the third exists

Stage	What you get	What breaks
Equation in a script	One answer, once	The next beam means copy-paste, and now there are two versions.
Equation in a function	Re-usable, testable	It returns a bare float. Nothing records the clause, the assumptions, or the range it is valid over — so nothing can be checked or reported.
Equation in a module	A <code>CalcResult</code>	Nothing. This is the shape everything in <code>design/</code> returns.

Every public calculation in this library returns a `CalcResult`, never a bare float, carrying six things:

.inputs	Echoed verbatim, with units
.basis	The clauses, in the order they were applied
.envelope	Validity limits, and whether you are inside them
.outputs	The results, with units
.checks	Pass/fail against each code criterion
.provenance	Version, author, checker, verification status

That is why the report in part 11 needs no assembly: the results already carry everything a report has to show.

Envelope versus check

The distinction that makes the library safe, and the one spreadsheets almost never make. Ask for 150 MPa concrete and the module *refuses*; ask for 100 and it answers.

```
python
>>> from austruct.materials import concrete
>>> concrete(150)
austruct.core.exceptions.OutsideEnvelope: Inputs outside validity envelope [Concrete properties (AS 3600:2018)]: 20 <= f'c <= 100 MPa [actual 150] OUTSIDE
>>> concrete(100)
Concrete(fc=100.0, density=2400.0, standard=Standard(code='AS 3600', edition=2018, title='Concrete structures', amendments=()), fcmi=99.0, fcm=110.0, Ec=42200.0, ec_source='table')
```

Screenshot 4.3 — an exception, not a number. The message names the limit, the clause behind it, and your value.

	Envelope	Check
Bounds	Where the method is valid	A code criterion
Breach means	The answer is <i>unknown</i>	The answer is <i>known and it fails</i>
You get	An exception, no number	A full result that reads FAIL

An overstressed beam is a good calculation with a failing result — you want to see it, with the working, so you can size up. A beam of 150 MPa concrete is not a calculation this library can do at all.

One import rule keeps it composable

Modules import from layers *above* them in this list, never below. That single rule is why you can call any of it in any order without circular surprises.

L0	core/	The contract: basis, envelope, provenance, units
L1	materials/, project/	Grades, bar catalogue, job record
L2	sections/, design_documentation/	Geometry, the designation grammar
L3	analysis/, loads/, structures/	Solver, combinations, envelopes
L4	design/	rc_common/ holds the mechanics; as3600/, as5100_5/, as4100/, as3700/ hold only what differs
L5	report/	The audit artifact

What is already there, and how to find it

Fifty-six modules are registered. `CATALOGUE.md` in the repository root lists every one with its clause coverage and verification status, and it is generated from the code itself so it cannot go stale. For a single package, ask Python:

```
python

>>> from austruact.design import as3600
>>> [n for n in dir(as3600) if not n.startswith("_")]
['CostRates', 'DevelopmentLength', 'OptimisationResult', 'SearchBounds', 'SectionCost', 'SizeRange',
'cd_dimension', 'check_bar_fit', 'check_crack_control', 'check_deflection', 'check_detailing',
'check_flexure', 'check_shear', 'check_span_to_depth', 'compression_development_length', 'constants',
'curtailment_extension', 'detailing', 'development_length', 'effective_shear_depth',
'effective_stiffness', 'flexure', 'ief_max', 'lap_length', 'minimum_clear_spacing',
'minimum_cost_section', 'minimum_shear_reinforcement', 'minimum_steel_area', 'moment_capacity',
'optimise', 'redistribution_limit', 'required_shear_reinforcement', 'required_steel_area',
'serviceability', 'shear', 'shear_capacity']
>>> print(as3600.check_shear.__doc__[900])
Check a section's shear capacity against a design shear force.

    Basis
    ----
    As :func:`shear_capacity`, plus ``V* <= phi.V_u`` (Cl 2.2.2).

    Parameters
    -----
    V_star:
        Design shear force (N), magnitude. Take it from
        :meth:`~astruact.analysis.results.BeamResults.shear_at_d_from_support`
        where the support introduces compression, or from
        :attr:`~astruact.analysis.results.BeamResults.max_shear` otherwise.
    M_star:
        Coexisting design moment (N.mm). Used by the general method only.

    Returns
    -----
    CalcResult
```

Screenshot 4.4 — every public calculation in AS 3600, listed by the library itself. The docstring names the Basis, the Parameters and the Returns. `dir(x)` is also the fastest recovery when you guess an attribute name wrong.

YOUR JOB IS THE MIDDLE STEP

You are not writing these equations and you are not re-deriving them. You are **reading the Basis, checking the constants against your copy of the Standard, and then calling the module** — after which every job that calls it inherits that one act of checking. That is the entire economic argument for the structure.

5 File one — the job (ASET component 2)

Write this completely before touching the member file. It holds no capacities. If you later design B2, B3 and the transfer slab, they import this same file and all move together when the client changes the plant weight.

`Project` is a plain record of client-supplied facts that quietly does real work: the occupancy drives the ψ combination factors, and the structure type decides whether you get the AS/NZS 1170.0 building set or the AS 5100.2 bridge set. Stating "office building" once is what stops a bridge combination reaching a floor beam.

```
job/job_data.py
23 # 1. The job record -- client-supplied facts, written down once
24 # -----
25 PROJECT = Project(
26     job_number="25-0142",
27     job_name="Northbank Depot",
28     client="Northbank Logistics Pty Ltd",
29     site_address="14 Kembla Street, Wollongong NSW",
30     jurisdiction="NSW",
31     structure_type=StructureType.BUILDING,
32     occupancy=Occupancy.OFFICE,
33     exposure=ExposureClassification.B1,
34     design_life_years=50,
35     importance_level=2,
36     engineer="A. Morrison",
37     checker="",
38 )
39
40 # -----
41
```

Screenshot 5.1 — note what is *not* here: no defaults invented on your behalf. An unstated design life stays empty so the report can say 'not stated'.

Spans and tributary widths get names, so every later use is self-evidently the same number. The library works in N, mm and MPa — which makes 1 kN/m exactly 1 N/mm — so areal loads are written in kPa the way a load schedule states them and converted exactly once, by multiplying by `kPa`.

job/job_data.py

```
41 # 2. Geometry the loads depend on
42 # -----
43 SPAN = 7.2 * m
44 """Centre-to-centre of the supporting blockwork corbels."""
45
46 TRIB_WIDTH = 3.6 * m
47 """Half the bay each side of B1 -- one-way slab spanning onto it."""
48
49 PLANT_POSITION = 3.0 * m
50 """Chainage of the condenser unit's nearer support foot, from grid A."""
51
52 # -----
53 # 3. Areal loads -> line loads on B1
54 # -----
55 # [ASSUMPTION] 180 mm one-way slab, 24 kN/m3 reinforced concrete.
56 G_AREAL = 4.32 * kPa + 0.90 * kPa
57 """Permanent action on the roof: 180 slab (4.32 kPa) + membrane, insulation and
58 services (0.90 kPa). Does NOT include beam self weight -- the library derives
59 that from the section, see 'SelfWeight' in beam_B1.py."""
60
61 Q_AREAL = 0.25 * kPa
62 """Imposed action, non-trafficable roof, AS/NZS 1170.1 Table 3.2."""
63
64 Q_PLANT = 40.0 * kN
65 """Rooftop condenser, operating weight, supplier data sheet NB-CD-04 Rev B."""
66
67
68 # -----
69
```

Screenshot 5.2 — the triple-quoted string under each value is a docstring, doing the job a cell comment does in a spreadsheet, except it cannot be scrolled out of view or lost in a copy-paste.

UNITS, ONCE

kN, m, kPa, kNm and kN_per_m are plain multipliers: they convert *into* N-mm on the way in (7.2 * m is 7200.0) and *out* on the way to a print statement (M / kNm). If a number is ever wrong by a factor of 1000, look here first.

Wind, to AS/NZS 1170.2

The library does not derive site wind — too many local judgements to hide inside a package. So it goes in your file, one multiplier per line, each tagged with its clause. This is the pattern to copy whenever you need something the library does not have.

job/job_data.py

```
69 # 4. Wind, to AS/NZS 1170.2
70 # -----
71 # [VECTOR] Every factor below is transcribed from the standard for THIS site
72 # and needs checking against the printed clause before issue.
73 V_R = 45.0
74 """Regional wind speed V_500, Region A4, AS/NZS 1170.2 Table 3.1 (m/s)."""
75
76 M_D = 1.00 # wind direction multiplier, Cl 3.3
77 M_ZCAT = 0.91 # terrain/height multiplier, TC2, z = 9.0 m, Table 4.1
78 M_S = 1.00 # shielding, Cl 4.3
79 M_T = 1.00 # topographic, Cl 4.4
80
81 C_PE = -0.90 # external pressure coefficient, roof, Table 5.3(A)
82 C_PI = 0.20 # internal pressure coefficient, Table 5.1(A)
83 K_A = 1.00 # area reduction, Cl 5.4.2
84 K_L = 1.00 # local pressure, Cl 5.4.4
85 C_DYN = 1.00 # dynamic response, Cl 6.1
86
87 AIR_DENSITY = 1.2e-6
88 """1.2 kg/m3 expressed in the library's N-mm system, so that
89 0.5 * rho * V^2 comes out in MPa (= N/mm2 = kPa/1000)."""
90
91
92 def design_wind_speed() -> float:
93     """V_des, theta to AS/NZS 1170.2 Cl 2.3, in m/s.
94
95     [BASIS] AS/NZS 1170.2 Cl 2.3: V_sit = V_R * M_d * (M_z,cat M_s M_t)
96     """
97     return V_R * M_D * (M_ZCAT * M_S * M_T)
98
99
100 def net_uplift_pressure() -> float:
101     """Net upward pressure on the roof, in the library's units (MPa).
102
103     [BASIS] AS/NZS 1170.2 Cl 2.4.1: p = 0.5 rho_air V_des^2 C_fig C_dyn
104     with C_fig = C_p,e K_a K_c K_l K_p - C_p,i (Cl 5.2)
105
106     Returned POSITIVE for uplift; the sign convention is applied at the call
107     site in beam_B1.py, where the load direction is visible next to the beam.
108     """
109     v_des = design_wind_speed()
110     c_fig = (C_PE * K_A * K_L) - C_PI # negative = suction/uplift
111     p = 0.5 * AIR_DENSITY * v_des**2 * c_fig * C_DYN
112     return abs(p)
113
114
115 # -----
116
```

Screenshot 5.3 — when the checker asks where $M_{z,cat} = 0.91$ came from, the answer is on the same line as the number. [VECTOR] marks values transcribed from a Standard, so every one can be found by text search.

The file's actual output is three numbers — `w_g`, `w_q` and `w_wu`, in the library's units. Everything above exists to produce them.

6 File two — the member

The import block answers the question this tutorial exists for.

```
job/beam_B1.py
15 from pathlib import Path
16
17 from austruct.analysis import UDL, PointLoad, SelfWeight, analyse_combinations, simply_supported
18 from austruct.core.units import kN, kN_per_m, kNm, kPa, m
19 from austruct.design import as3600
20 from austruct.design_documentation import designate, parse
21 from austruct.loads import ActionType, LoadCase
22 from austruct.materials import describe_options
23 from austruct.report import HtmlRenderer, MarkdownRenderer, Report, plots
24
25 # Your own job file, sitting next to this one. Same import machinery, no
26 # package install -- Python looks in the script's own directory first.
27 import job_data as job
28
```

Screenshot 6.1 — six imports from the installed library, then one from the file next door. The blank line between them is worth keeping: above it, code you did not write; below it, code you did.

Read down the imports and you can predict the calculation before reading a line of it. Imports at the top of a file are a table of contents.

The design decision, as one line of text (ASET component 4)

```
job/beam_B1.py
30 # 1. The design decision, written the way it goes on the drawing
31 # -----
32 DESIGNATION = "350 x 650 | C32 | COV 40 | BOT 3-N24 | LIG N12-2L@250"
33 section = parse(DESIGNATION, name="B1")
34
35 assert designate(section) == DESIGNATION, "designation does not round-trip"
36
37 print("SECTION")
38 for line in section.describe():
39     print("    ", line)
40
41 # -----
42
```

Screenshot 6.2 — `parse()` in, `designate()` back out. The `assert` is a one-line guard that the round trip is exact.

Field	Means	Rules
350 x 650	b × D, mm	Always first
C32	f _c = 32 MPa	Keyword-led and order-independent. An unrecognised field raises rather than being ignored — silently dropping TOP 2-N16 would hand you a singly reinforced capacity for a doubly reinforced beam.
COV 40	Cover to the fitment, mm	
BOT 3-N24	Three N24 bottom bars	
LIG N12-2L@250	N12 two-leg ligatures at 250	

Why a string and not five arguments: it is the thing everyone else already reads. It goes in the schedule, on the drawing, in the email to the drafter, and into `parse()` unchanged — so there is no transcription step, and therefore no transcription error.

```
python beam_B1.py — first block

SECTION
  Section      = B1
  D            = 650 mm
  A_g          = 227500 mm^2
  y_c          = 325.0 mm from top
  I_g          = 8.01e+09 mm^4
  Z_bottom    = 2.465e+07 mm^3
  b_v         = 350 mm

Concrete:
  f'c          = 32 MPa
  density      = 2400 kg/m^3
  Ec           = 30100 MPa (source: table)
  f'ct         = 2.04 MPa
  f'ct.f       = 3.39 MPa
  alpha_2     = 0.850
  gamma       = 0.826
  epsilon_cu  = 0.0030

Longitudinal reinforcement:
  3-N24 at d = 586 mm (bottom main)

Fitments:  N12-2leg @ 250
```

Screenshot 6.3 — what that one line expanded into. $d = 586$ mm is derived from cover + fitment + half a bar, not typed: one of the most common arithmetic slips in hand calculations, done once here in tested code.

7 Load cases and combinations (AS/NZS 1170.0)

A load case says *what the loads are and which action they represent*. A combination says *how the jurisdiction adds them up*. Keeping them separate is why you never write `1.2 * dead + 1.5 * live` by hand again.

```
job/beam_B1.py
42 # 2. Load cases -- unfactored, each tagged with the action it represents
43 # -----
44 cases = (
45     LoadCase(
46         "G_roof",
47         ActionType.G,
48         (
49             UDL(magnitude=job.W_G),
50             SelfWeight(area=section.geometry.area, density=2400.0, length=job.SPAN),
51         ),
52     ),
53     LoadCase(
54         "Q_roof",
55         ActionType.Q,
56         (
57             UDL(magnitude=job.W_Q),
58             PointLoad(position=job.PLANT_POSITION, magnitude=job.Q_PLANT),
59         ),
60     ),
61     LoadCase(
62         "Wu_uplift",
63         ActionType.Wu,
64         # [UNITS] Upward is negative in this package's sign convention.
65         (UDL(magnitude=-job.W_WU),),
66     ),
67 )
68
```

Screenshot 7.1 — each case carries an `ActionType`, and that tag is the mechanism: it connects ‘this 18.79 kN/m is permanent action’ to ‘the code multiplies permanent action by 1.2 in this combination’.

- **SelfWeight instead of a number** — derived from the section's own area, so it follows when you change 350×650 to 400×750 . A hand-computed 5.36 would not.
- **A UDL and a point load in one case** — the roof pressure and the plant unit are both Q and must be factored together.
- **The minus sign on the wind UDL** — downward is positive throughout, so uplift is negative, applied here next to the beam rather than buried in the job file.

```
python beam_B1.py

LOAD CASES (unfactored)
  G_roof    G    UDL      18.79 kN/m
  G_roof    G    SelfWeight 5.36 kN/m
  Q_roof    Q    UDL      0.90 kN/m
  Q_roof    Q    PointLoad 40.0 kN at 3.0 m
  Wu_uplift Wu    UDL     -3.98 kN/m
```

Screenshot 7.2 — every load echoed with its type and intensity. Reading this before looking at any capacity is the equivalent of checking your load take-down first.

You never type a load factor. `PROJECT.load_combinations(sls=False)` reads the structure type and occupancy off the record from part 5 and returns the AS/NZS 1170.0 Cl 4.2.2 ultimate set with the ψ factors already resolved.

```
python

>>> from austruct.project import Occupancy, Project, StructureType
>>> p = Project(job_number="25-0142", occupancy=Occupancy.OFFICE,
...            structure_type=StructureType.BUILDING, engineer="A. Morrison")
>>> p.combination_factors()
(0.4, 0.7, 0.4)
>>> for c in p.load_combinations(sls=False):
...     print(" ", c)
  ULS1: 1.35G
  ULS2: 1.2G + 1.5Q
  ULS3: 1.2G + 0.6Q (long term)
  ULS4: 1.2G + Wu + 0.4Q
  ULS5: 0.9G + Wu (uplift/reversal)
  ULS6: G + Eu + 0.4Q
  ULS7: 1.2G + Su + 0.4Q
```

Screenshot 7.3 — `(0.4, 0.7, 0.4)` is (ψ_c, ψ_s, ψ_l) for an office, arriving in ULS3 and ULS4. Change `Occupancy.OFFICE` to `STORAGE` and every combination changes with it.

COMBINATIONS YOU DID NOT ASK FOR ARE A FEATURE

Seven come back, including earthquake and snow, because the *project* implies them; the analysis drops the ones no supplied case contributes to, so they cost nothing. The point is that the list comes from the jurisdiction, not from what you remembered on the day — and the one you forget is the one that governs.

8 Analyse, then envelope (ASET component 3)

Two lines: build the member, then run every combination over it and keep the worst of each action — with a record of which combination caused it.

job/beam_B1.py

```
82 # 3. Analyse over every ULS combination the project record implies
83 # -----
84 combinations = job.PROJECT.load_combinations(sls=False)
85
86 print("\nCOMBINATIONS (AS/NZS 1170.0 Cl 4.2.2, psi factors from the occupancy)")
87 for combo in combinations:
88     print("    ", combo)
89
90 beam = simply_supported(job.SPAN, section=section, name="B1")
91 env = analyse_combinations(bean, cases, combinations)
92
93 print("\nENVELOPE")
94 print(env.summary())
95
96 # -----
97
```

Screenshot 8.1 — `simply_supported()` is one of a family (`cantilever` , `propped` , `continuous`). Passing `section=` rather than a bare EI means deflections use the section you detailed.

```
python beam_B1.py

ENVELOPE
Envelope: B1 [ULS]
7 combination(s) analysed

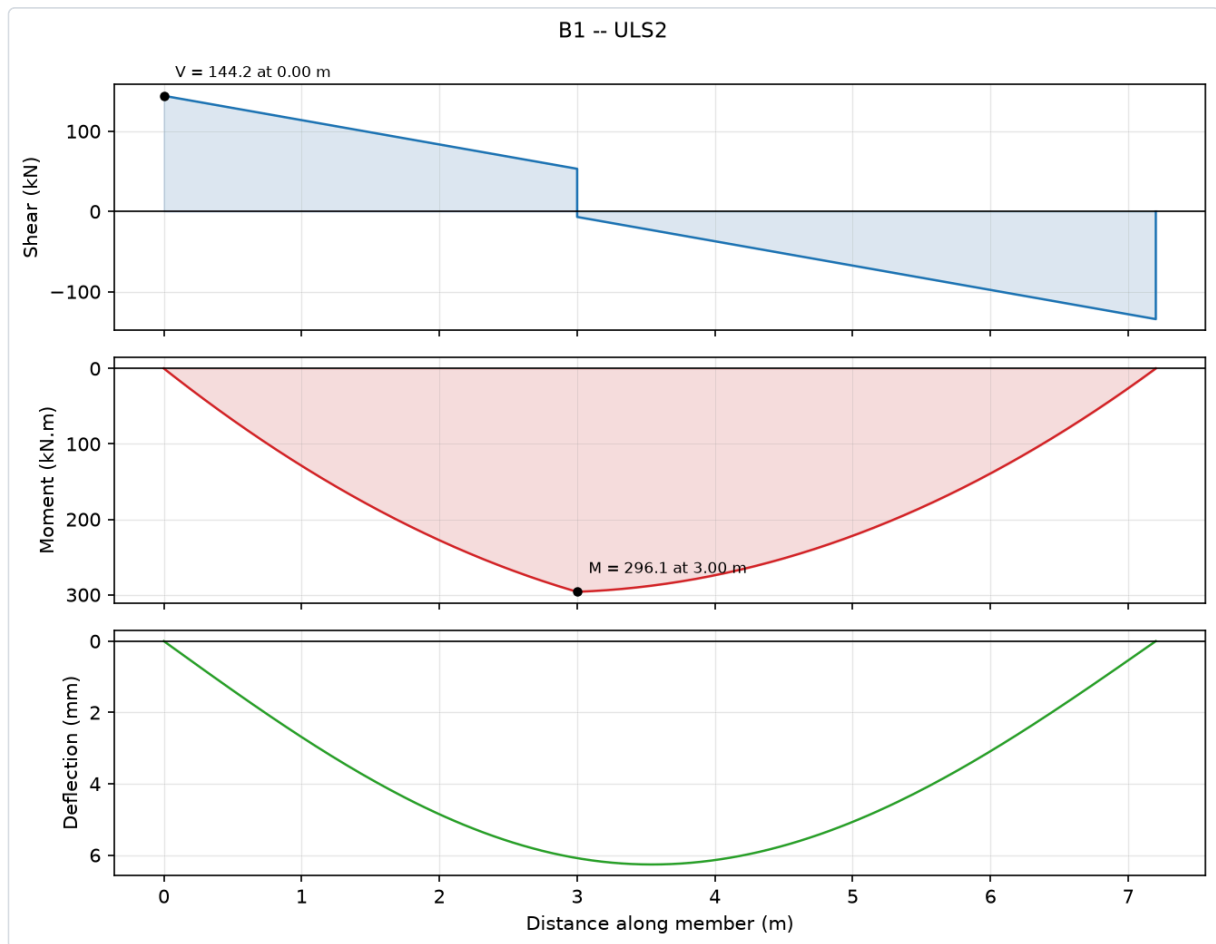
ULS1: 1.35G
ULS2: 1.2G + 1.5Q
ULS3: 1.2G + 0.6Q (long term)
ULS4: 1.2G + Wu + 0.4Q
ULS5: 0.9G + Wu (uplift/reversal)
ULS6: G + Eu + 0.4Q
ULS7: 1.2G + Su + 0.4Q

Governing actions:
M_max (sagging)      296.07 kN.m at x = 3.000 m [ULS2]
M_min (hogging)      0.00 kN.m at x = 0.000 m [ULS1]
V_max               144.18 kN at x = 0.000 m [ULS2]
V_min              -134.18 kN at x = 7.200 m [ULS2]
Deflection max       6.26 mm at x = 3.538 m [ULS2]

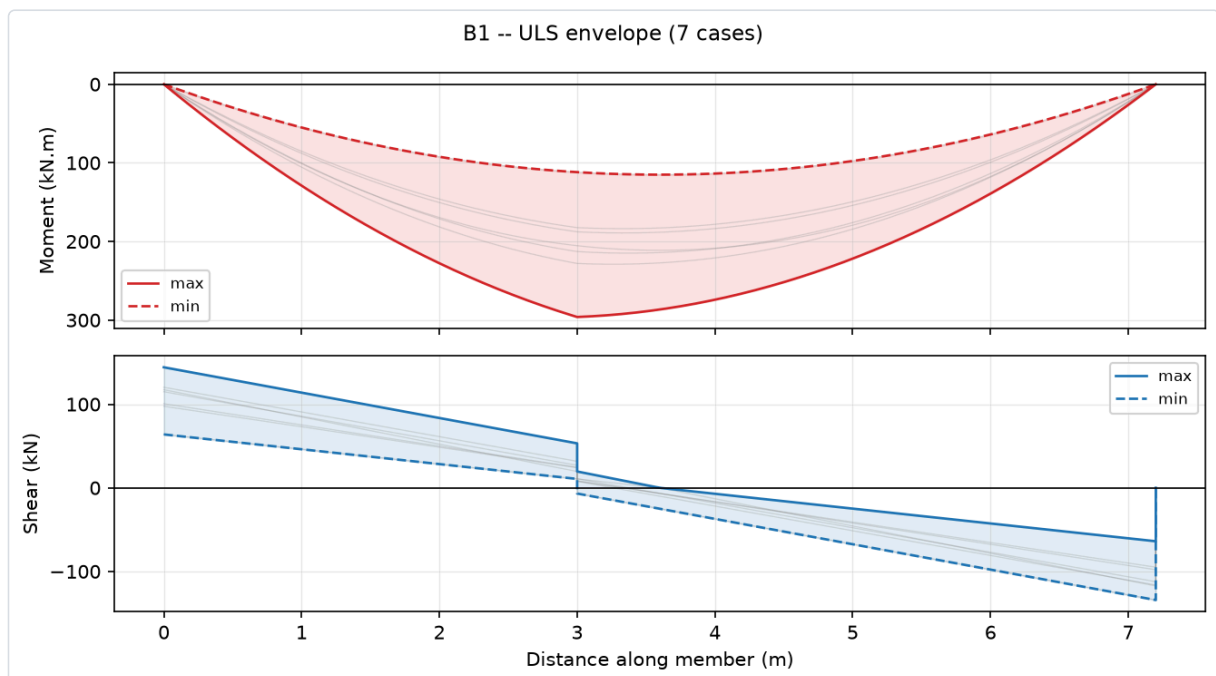
Reactions (positive upward):
A      max    144.18 kN [ULS2]   min    63.90 kN [ULS5]
B      max    134.18 kN [ULS2]   min    63.90 kN [ULS5]
```

Screenshot 8.2 — every row carries the combination that caused it in square brackets, which is the most useful thing an envelope can tell you.

- **M* = 296.1 kN.m at x = 3.000 m [ULS2]** — under the plant load, from 1.2G + 1.5Q. Not at midspan, because the point load is not at midspan.
- **M_{hog} = 0** — no reversal under any of the seven.
- **Minimum reaction 69.43 kN [ULS5]**, still downward: no hold-down required. A real check, which fell out of the envelope for free.
- **Deflection 6.26 mm** — from *factored* loads on a gross section, so it is a solver output, not a serviceability check.



Screenshot 8.3 — the governing combination alone. Sagging is drawn downward, so the moment diagram hangs the way the beam does. The shear step at 3.0 m is the 60 kN factored plant load; $V = 144.2$ kN at the support, 126.4 kN at d.



Screenshot 8.4 — the envelope, with every contributing combination faint behind it. The dashed *min* curve still sits near 90 kN.m of sagging at midspan: that is the picture of ‘no reversal anywhere’, and it is worth more than reading $M_{hog} = 0$.

9 Flexure and shear to AS 3600:2018

After all that, the design is three lines. That is the right proportion: the hard part is getting the right demand to the right section.

```
job/beam_B1.py
97 # 4. Design verification to AS 3600:2018
98 # -----
99 M_star = env.M_star
100 V_star = env.shear_at_d_from_support(section.d)
101
102 flexure = as3600.check_flexure(section, M_star)
103 shear = as3600.check_shear(section, V_star=V_star, M_star=M_star)
104
```

Screenshot 9.1 — `shear_at_d_from_support(d)` is the code allowance for taking design shear at d from a support that introduces compression. It is a method on the envelope because only the envelope knows the shear diagram.

```
python beam_B1.py

AS 3600:2018 CHECKS
M* = 296.1 kN.m [ULS2] phi.Muo = 317.4 kN.m util 0.933 PASS
V* = 126.4 kN [ULS2] phi.Vu = 339.5 kN util 0.372 PASS

Reversal check: M*_hogging = 0.0 kN.m -> no reversal, section as detailed
```

Screenshot 9.2 — both checks with the governing combination, capacity, utilisation and verdict.

Note that `check_shear` takes the coexisting moment as well as the shear. That is not padding: AS 3600:2018 uses a modified compression field formulation in which the concrete contribution depends on longitudinal strain, which depends on M^* . Shear capacity is *coupled to the demand* — exactly the case Ferster names as the hard one, and the reason you want it inside a tested module.

Five checks, not two

Check	Clause	Here
Ductility, $k_{u0} \leq 0.36$	CI 8.1.5	0.147 — PASS
Minimum strength, $M_{u0} \geq 1.2 M_{cr}$	CI 8.1.6.1	$373 \geq 110$ kN.m — PASS
$M^* \leq \phi M_{u0}$	CI 2.2.2	$296.1 \leq 317.4$ kN.m — PASS
Minimum shear reinforcement	CI 8.2.1.7	$0.905 \geq 0.317$ mm ² /mm — PASS
$V^* \leq \phi V_u$	CI 8.2.1.1	$126.4 \leq 339.5$ kN — PASS

The ductility check is the one a hand calculation most often skips, and the minimum-fitment check is not decoration: the simplified method's $k_v = 0.15$ is only available *because* the section carries at least minimum fitments. Widen the ligatures past that and both change together.

```

python

>>> from austruct.core.units import kNm
>>> from austruct.design import as3600
>>> from austruct.design_documentation import parse
>>> section = parse("350 x 650 | C32 | COV 40 | BOT 3-N24 | LIG N12-2L@250")
>>> flexure = as3600.check_flexure(section, 296.1 * kNm)
>>> for ref in flexure.basis:
...     print(" ", ref)
    AS 3600:2018 Cl 8.1.2 -- Ultimate strength in bending
    AS 3600:2018 Cl 8.1.3 -- Rectangular stress block
    AS 3600:2018 Table 2.2.2 -- Capacity reduction factors
    AS 3600:2018 Cl 8.1.5 -- Ductility limit on k_uo
    AS 3600:2018 Cl 8.1.6.1 -- Minimum strength requirement
>>> for c in flexure.checks:
...     print(f"    {c.label:<34} {c.actual / c.display_factor:9.2f} {c.operator} "
...           f"{c.limit / c.display_factor:9.2f} {c.display_unit or c.unit}")
    Ductility, k_uo                0.15 <=      0.36 -
    Minimum strength, M_uo >= 1.2 M_cr  373.47 >=    109.57 kN.m
    Flexural strength, M* <= phi.M_uo  296.10 <=    317.45 kN.m
>>> for lim in flexure.envelope.limits:
...     print(f"    {lim.name:<10} {lim.value:8.0f} {lim.unit:<8} allowed {lim.lower:g} to
{lim.upper:g}")
    f'c                32 MPa        allowed 20 to 100
    f'c                32 MPa        allowed 20 to 100
    density            2400 kg/m^3    allowed 1800 to 2800
>>> for note in flexure.envelope.notes:
...     print("    -", note)
    - Non-prestressed reinforced concrete only. No axial force.
    - Sagging bending with the compression face at the top of the section as supplied. Model hogging
by inverting the section.
    - Torsion, lateral instability of slender beams and the effects of sustained load are not
considered.
>>> flexure.provenance.module
'austruct.design.as3600.flexure'
>>> flexure.provenance.status.value
'UNVERIFIED'

```

Screenshot 9.3 — the audit trail, interrogated from the REPL: the clauses in application order, the checks, the envelope that was enforced, and the module's verification status. This is the `CalcResult` from part 4, in use.

READ THE ENVELOPE NOTES THE FIRST TIME YOU USE A MODULE

They say what it did *not* do: no axial force, no torsion, no lateral instability, and *sagging only* — "model hogging by inverting the section". That last is a real trap on a continuous beam, and the library will not warn you twice.

outputs/B1_report.html

WORKING

Symbol	Description	Value	Unit
d_v	Effective shear depth	527.4	mm
d_o	Depth to outermost tensile bar	586	mm
(A_sv/s)_min	Minimum shear reinforcement	0.3168	mm^2/mm
k_v	Concrete shear factor (simplified, >= minimum fitments)	0.15	-
theta_v	Compression strut angle	36	deg
V_uc + V_us	Before the V_u,max cap	485	kN

RESULTS

Symbol	Description	Value	Unit
V_uc	Concrete contribution	156.6	kN
V_us	Fitment contribution	328.4	kN
V_u,max	Web crushing limit	1545	kN
V_u	Nominal shear capacity	485	kN
phi	Capacity reduction factor, shear	0.7	-
phi.V_u	Design shear capacity	339.5	kN

CHECKS

Criterion	Actual		Limit	Unit	Utilisation	Status
Minimum shear reinforcement, A_sv/s	0.9048	>=	0.3168	mm^2/mm	0.350	PASS
Shear strength, V* <= phi.V_u	126.4	<=	339.5	kN	0.372	PASS

astruct.design.as3600.shear v0.1.0 · UNVERIFIED · not checked · run 2026-09-03T18:41:37+00:00

B1

Screenshot 9.4 — the shear check as it lands in the report: inputs, seven clauses, validity envelope, working, results, checks. $d_v = 527.4$ mm, $k_v = 0.15$ and $\theta_v = 36^\circ$ are reported rather than assumed.

AS 5100.5 IS NOT ‘AS 3600 WITH DIFFERENT NUMBERS’;

For flexure the two give an identical nominal M_{u0} and differ only in ϕ . For **shear** they share no code at all: AS 5100.5:2017 predates the 2018 revision and uses the $\beta_1\beta_2\beta_3 / f_{cv} / d_o$ family. Swapping `as3600` for `as5100_5` in a shear line and expecting a small change is the most likely way to get a bridge wrong with this toolkit.

10 The loop: FAIL is a result, not an error

The section here was the second try. Here is the first, and the thirty seconds it took to fix.

```
python
>>> from austruct.core.units import kNm
>>> from austruct.design import as3600
>>> from austruct.design_documentation import parse
>>> M_star = 296.1 * kNm
>>> trial = parse("350 x 650 | C32 | COV 40 | BOT 3-N20 | LIG N12-2L@250")
>>> first = as3600.check_flexure(trial, M_star)
>>> print(f"{first.utilisation:.3f} {'PASS' if first.passed else 'FAIL'}")
1.312 FAIL
>>> as3600.required_steel_area(trial, M_star).get("Ast_req")
1255.2780456542969
>>> trial.Ast
942.4777960769379
>>> better = parse("350 x 650 | C32 | COV 40 | BOT 3-N24 | LIG N12-2L@250")
>>> second = as3600.check_flexure(better, M_star)
>>> print(f"{second.utilisation:.3f} {'PASS' if second.passed else 'FAIL'}")
0.933 PASS
```

Screenshot 10.1 — 3-N20 gives 1.312, FAIL. `required_steel_area` says the section needs 1255 mm² and has 942. 3-N24 gives 1357 mm² and 0.933, PASS. No spreadsheet was reopened and no clause re-read.

Three things make that fast, and all three are choices worth copying: a failing check still returns a *full* result rather than an exception; the inverse problem (`required_steel_area`) is a first-class function, so you are not bisecting by hand; and the decision is one string, so changing `BOT 3-N20` to `BOT 3-N24` is the entire edit — d, self weight, diagrams and report all follow.

```
python beam_B1.py

Bar arrangements that would satisfy the required area:
Required As = 1260 mm^2

Arrangement      As prov      Excess
-----
12-N12            1357          7.7%
3-N24             1357          7.7%
7-N16             1407         11.7%
5-N20             1571         24.7%
2-N32             1608         27.7%
```

Screenshot 10.2 — `describe_options` takes the required area and returns every practical arrangement with its excess. 3-N24 at 7.7% over is the obvious pick; 12-N12 provides the same area and would not fit.

Once you are running that loop more than a few times, `as3600.minimum_cost_section` will do it for you — same checks, same clauses, driven by a search over widths, depths and bar arrangements against concrete and steel rates. That only becomes possible because the check was written as a function.

```
bash

(.venv) $ cd ~/jobs/25-0142-northbank          # one level above the script
(.venv) $ python job/beam_B1.py                 # with a bare OUT = "../outputs"
...analysis and design output, as normal...
Traceback (most recent call last):
  File "../job/beam_B1.py", line 137, in <module>
    plots.save(fig_dia, f"{OUT}/B1_diagrams.png")
    ... 9 frames inside matplotlib and PIL omitted ...
FileNotFoundError: [Errno 2] No such file or directory: '../outputs/B1_diagrams.png'
```

Screenshot 10.3 — the third failure mode, after the two in part 2: the shipped file with its output line replaced by a bare `OUT = "../outputs"`, run one directory up. The imports resolve (Python adds the *script's* folder to the path, not yours) but the relative output path does not. The file ships with `Path(__file__).resolve().parent.parent / "outputs"` instead.

11 The report (ASET component 6)

A calculation that only exists in a terminal is not a deliverable. The last block of the job file assembles the document, and the assembling is mostly just adding the results you already have, in reading order.

```
job/beam_B1.py
146 # 6. The report
147 # -----
148 report = Report(
149     title=f"Roof beam B1 -- {job.PROJECT.job_name}",
150     signature=job.PROJECT.signature_block(element="Roof beam B1", revision="A"),
151     preamble=(
152         f"Simply supported roof beam, {job.SPAN / m:.1f} m clear span, {DESIGNATION}. "
153         f"Demands enveloped over {len(env.combinations)} ultimate combinations to "
154         f"AS/NZS 1170.0; capacities to AS 3600:2018."
155     ),
156 )
157 report.add_scope(
158     "Flexural and shear strength of B1 at the ultimate limit state only. "
159     "Deflection, crack control, anchorage and the supporting blockwork are "
160     "outside this calculation."
161 )
162 report.add_assumption(
163     f"Permanent action {job.G_AREAL / kPa:.2f} kPa over a {job.TRIB_WIDTH / m:.1f} m "
164     f"tributary width, plus beam self weight at 24 kN/m3. Imposed action "
165     f"{job.Q_AREAL / kPa:.2f} kPa (AS/NZS 1170.1 Table 3.2, non-trafficable roof) "
166     f"with a {job.Q_PLANT / kN:.0f} kN plant load at {job.PLANT_POSITION / m:.1f} m. "
167     f"Ultimate wind uplift {job.W_WU / kN_per_m:.2f} kN/m from "
168     f"V_des = {job.design_wind_speed():.1f} m/s to AS/NZS 1170.2."
169 )
170 report.add(env.to_calc_result())
171 report.add_figure(f"Shear, moment and deflection under the governing combination "
172                  f"({governing_combo.name})", "B1_diagrams.png")
173 report.add_figure("Moment and shear envelopes over all ULS combinations", "B1_envelope.png")
174 report.add(flexure)
175 report.add(shear)
176 report.add_figure("Section as detailed", "B1_section.png")
177 report.add_conclusion(
178     f"B1 as detailed ({DESIGNATION}) is adequate in flexure "
179     f"(utilisation {flexure.utilisation:.2f}) and in shear "
180     f"(utilisation {shear.utilisation:.2f}) at the ultimate limit state."
181 )
```

Screenshot 11.1 — a title, a signature block generated from the project record, then `add()` calls in order. The results carry their own inputs, clauses, working and checks, so nothing is transcribed.

`add_scope`, `add_assumption` and `add_conclusion` are different: they take prose, and the library cannot generate them. A report read by an independent reviewer needs to say what was assumed and what the numbers mean, and keeping that narrative in the same ordered list as the calculations is what stops it drifting to the back of the document where nobody reads it.

outputs/B1_report.html

Roof beam B1 -- Northbank Depot

Job: 25-0142 Project: Northbank Depot Element: Roof beam B1 Revision: A

Result: PASS

NOT VERIFIED FOR ISSUE — one or more modules used in this report have not been checked against the printed standard. Development use only.

Simply supported roof beam, 7.2 m clear span, 350 x 650 | C32 | COV 40 | BOT 3-N24 | LIG N12-2L@250. Demands enveloped over 7 ultimate combinations to AS/NZS 1170.0; capacities to AS 3600:2018.

Scope

SCOPE

Flexural and shear strength of B1 at the ultimate limit state only. Deflection, crack control, anchorage and the supporting blockwork are outside this calculation.

Assumptions

ASSUMPTION

Permanent action 5.22 kPa over a 3.6 m tributary width, plus beam self weight at 24 kN/m3. Imposed action 0.25 kPa (AS/NZS 1170.1 Table 3.2, non-trafficable roof) with a 40 kN plant load at 3.0 m. Ultimate wind uplift 3.98 kN/m from V_des = 41.0 m/s to AS/NZS 1170.2.

1. Enveloped demands -- B1 (ULS)

INPUTS

Symbol	Description	Value	Unit
n	Combinations analysed	7	-

BASIS

- First principles — Linear elastic analysis, results enveloped over load combinations
- AS/NZS 1170.0:2002 Cl 4.2.2 — Combinations for ultimate limit states
- AS/NZS 1170.0:2002 Cl 4.2.2 — Combinations for ultimate limit states
- AS/NZS 1170.0:2002 Cl 4.2.2 — Combinations for ultimate limit states
- AS/NZS 1170.0:2002 Cl 4.2.2 — Combinations for ultimate limit states
- AS/NZS 1170.0:2002 Cl 4.2.2 — Combinations for ultimate limit states
- AS/NZS 1170.0:2002 Cl 4.2.2 — Combinations for ultimate limit states
- AS/NZS 1170.0:2002 Cl 4.2.2 — Combinations for ultimate limit states

RESULTS

Symbol	Description	Value	Unit
M*_sag	Governing sagging moment [ULS2]	296.1	kN.m
M*_hog	Governing hogging moment [ULS1]	0	kN.m
V*	Governing shear [ULS2]	144.2	kN
delta	Maximum deflection [ULS2]	6.263	mm
R_A	Maximum reaction [ULS2]	144.2	kN

Screenshot 11.2 — job header, overall verdict, the not-verified banner, then your preamble, scope and assumptions.

outputs/B1_report.html

Torsion, lateral instability of slender beams and the effects of sustained load are not considered.

WORKING

Symbol	Description	Value	Unit
α_2	Stress block intensity	0.85	-
γ	Stress block depth ratio	0.826	-
d_n	Neutral axis depth	86.3	mm
$\gamma \cdot d_n$	Stress block depth	71.28	mm
d	Effective depth	586	mm
d_o	Depth to outermost tensile layer	586	mm
k_u	d_n / d	0.1473	-
k_{uo}	d_n / d_o	0.1473	-
z	Lever arm	550.4	mm
C_c	Concrete compressive force	6.786e+05	N
T_s	Tensile force in reinforcement	6.786e+05	N
A_{st}	Tensile steel area	1357	mm ²
M_{cr}	Cracking moment	91.31	kN.m

RESULTS

Symbol	Description	Value	Unit
ϕ	Capacity reduction factor (Class N)	0.85	-
M_{uo}	Nominal moment capacity	373.5	kN.m
$\phi \cdot M_{uo}$	Design moment capacity	317.4	kN.m

CHECKS

Criterion	Actual		Limit	Unit	Utilisation	Status
Ductility, k_{uo}	0.1473	<=	0.36	-	0.409	PASS
Minimum strength, $M_{uo} \geq 1.2 M_{cr}$	373.5	>=	109.6	kN.m	0.293	PASS
Flexural strength, $M^* \leq \phi \cdot M_{uo}$	296.1	<=	317.4	kN.m	0.933	PASS

astruct.design.as3600.flexure v0.1.0 · UNVERIFIED · not checked · run 2026-09-03T18:41:37+00:00

Screenshot 11.3 — the flexure check as issued: every intermediate from α_2 through d_n , k_{uo} and the lever arm, then the three checks with utilisations. A reviewer can follow this without opening Python.

```
python beam_B1.py — last lines

REPORT all checks pass: True    issuable: False
       figures and report written to outputs/
```

Screenshot 11.4 — the two flags that matter.

PASSED AND ISSUABLE ARE DIFFERENT QUESTIONS

passed asks: did the beam satisfy the criteria? **issuable** asks: has the code that evaluated those criteria been checked against the printed Standard? A calculation can pass and still not be issuable — exactly the state this library is in today — and the report says so on its own face.

The HTML renderer writes a self-contained page: print it to PDF from any browser, or with `chrome --headless --print-to-pdf=B1_report.pdf outputs/B1_report.html`. The Markdown renderer suits the other route — paste into a Word template, or check the `.md` into git and diff calculations between revisions.

12 What to do next

The habits worth carrying over

- 1 **One job folder, two files, run from inside it.** Copy the folder for the next job rather than starting blank.
- 2 **Job data first, completely.** Loads that live in one place cannot disagree with themselves.
- 3 **Never type a load factor.** If you are writing `1.2 *` anything, the project record is missing information it should have.
- 4 **The design decision is one string** — the same string as the drawing.
- 5 **Read the Basis and the envelope notes before you trust a module**, and validate its constants against your copy of the Standard. That check is inherited by every job that calls it.
- 6 **Write the scope, assumptions and conclusion yourself.** The library can generate everything except the engineering.

The obvious extensions to this job

Next	Call
Serviceability — deliberately skipped here	<code>PROJECT.load_combinations(uls=False)</code> , then <code>as3600.effective_stiffness</code> and <code>as3600.check_deflection</code>
Crack control and detailing	<code>as3600.check_crack_control</code> , <code>as3600.check_detailing</code> , <code>as3600.check_bar_fit</code>
B2 and B3 as well	A second script importing the same <code>job_data</code> , or a CSV member schedule through <code>design_documentation</code>
Let a search pick the section	<code>as3600.minimum_cost_section</code>
Continuous beams	<code>austruct.analysis</code> pattern loading and redistribute — and invert the section for hogging
Steel, masonry, bridges	<code>design.as4100</code> , <code>design.as3700</code> , <code>design.as5100_5</code> — same contract, same call shape

THE STANDING CAVEAT

Nothing in this library has been verified against the printed Standards. Every report it produces says so on its face, and this tutorial does not change that. Treat every number here as an illustration of a workflow, not as a design.